

Probabilistic Machine Learning

Lecture 07: Selected Probabilistic Algorithms

Prof. Dr. Michael Granitzer

Module 1

Hard Assignments First

Guiding Question: *k-means gives every point exactly one cluster. What is lost for a point that sits between two?*

Learning Without a Target

Supervised

A **target concept** is given: each example carries a label, and the objective is to reproduce it on new data.

Unsupervised

No target concept. The data carries no labels, and the objective must come from the structure itself.

Cluster analysis is the unsupervised classification of a set of objects into groups such that objects **within** a group are similar and objects in **different** groups are dissimilar.

The Stages of a Cluster Analysis

Represent

Choose the attributes and their scaling.

Measure

Choose a distance or similarity.

Group

Run an algorithm that optimises some criterion.

Interpret

Decide whether the groups mean anything.

Every clustering algorithm **always returns clusters**, including on data that has none. The output is never evidence that structure exists.

Clustering has no ground truth. The **representation and the distance encode what "similar" means** — the algorithm only optimises the criterion you handed it.

What a Distance Must Satisfy

A **metric** d satisfies, for all $\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3$:

1. $d(\mathbf{x}_1, \mathbf{x}_2) \geq 0$ — non-negativity
2. $d(\mathbf{x}_1, \mathbf{x}_1) = 0$ — identity
3. $d(\mathbf{x}_1, \mathbf{x}_2) = d(\mathbf{x}_2, \mathbf{x}_1)$ — symmetry
4. $d(\mathbf{x}_1, \mathbf{x}_3) \leq d(\mathbf{x}_1, \mathbf{x}_2) + d(\mathbf{x}_2, \mathbf{x}_3)$ — triangle inequality

The pairwise distances of n objects form the **distance matrix**, which is what most clustering algorithms actually consume.

Numeric Attributes: the Minkowski Family

$$d(\mathbf{x}_1, \mathbf{x}_2) = \left(\sum_{i=1}^p |x_{1i} - x_{2i}|^r \right)^{1/r}$$

$$r = 1$$

Manhattan — sum of absolute differences. Robust to single large deviations.

$$r = 2$$

Euclidean — straight-line distance. The default.

$$r \rightarrow \infty$$

Chebyshev — the largest single-attribute difference dominates.

Minkowski distances are **not scale-invariant**. An attribute measured in metres and one in millimetres do not contribute equally — standardise first, or the unit of measurement chooses your clusters.

Binary Attributes

With f_{11} the count of attributes present in both objects, f_{00} absent in both, and f_{01}, f_{10} present in exactly one:

Simple Matching Coefficient

$$\text{SMC} = \frac{f_{11} + f_{00}}{f_{11} + f_{00} + f_{01} + f_{10}}$$

Counts **shared absences** as agreement.

Jaccard Coefficient

$$J = \frac{f_{11}}{f_{11} + f_{01} + f_{10}}$$

Ignores shared absences.

For sparse data the difference is decisive: two shopping baskets agree on thousands of products **neither** contains. SMC would call them nearly identical Jaccard would not.

Mini-Exercise

Two documents are represented by which of 10,000 words they contain. Document A has 30 words, document B has 35, and they share 20.

1. Compute SMC and Jaccard.
2. Which is the honest similarity here?

Mini-Exercise — Solution

The distance measure **is** the definition of similarity. Scale sensitivity and the treatment of shared absences change the clustering before any algorithm runs.

Mini-Exercise — Solution

1. $f_{11} = 20$, $f_{01} + f_{10} = 25$, $f_{00} = 9955$.

$$\text{SMC} = \frac{9975}{10000} = 0.9975 \quad \cdot \quad J = \frac{20}{45} \approx 0.44$$

2. Jaccard. SMC reports 99.75% similarity for two documents that share fewer than half their words — the agreement is almost entirely about words neither uses.

The distance measure **is** the definition of similarity. Scale sensitivity and the treatment of shared absences change the clustering before any algorithm runs.

The Idea

Represent each cluster by one **exemplar** — a centroid — and assign every object to the nearest one. The quality criterion is the total squared distance to the assigned exemplar:

$$\text{SSE} = \sum_{j=1}^k \sum_{\mathbf{x} \in C_j} d(\mathbf{x}, \boldsymbol{\mu}_j)^2$$

Minimising SSE over all assignments is **NP-hard**. k-means is a heuristic that finds a local optimum — which is why its result depends on where it starts.

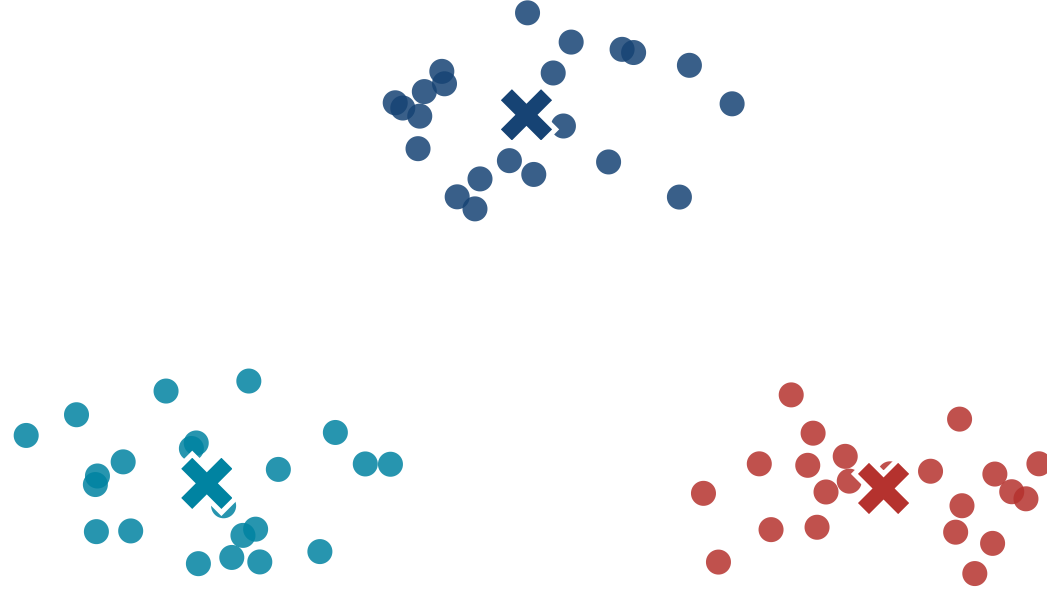
The Algorithm

k-means (Lloyd)

1. Choose k initial centroids.
2. **Assign** each object to the nearest centroid.
3. **Move** each centroid to the mean of its assigned objects.
4. Repeat 2–3 until the assignment stops changing.



Move each centroid to the mean of its points (round 3)



Properties

Fast

$O(nkp)$ per iteration, and few iterations in practice — it scales where HAC does not.

Converges

SSE decreases at every step and cannot increase, so the procedure terminates.

To a local optimum

The result depends on initialisation. Standard practice: restart several times and keep the best SSE.

k-medoids uses an actual data point as the exemplar instead of the mean. It is slower but works with any distance matrix — including where a "mean" is undefined.

Mini-Exercise

1. Why must k-means terminate?
2. Why is the mean the right update for the SSE criterion?
3. What would change if you used the **median** instead?

Mini-Exercise — Solution

k-means alternates **assign** and **move**, minimising SSE to a local optimum. The mean is not a convenience — it is the minimiser of the criterion, which is why changing the criterion changes the update.

Mini-Exercise — Solution

1. Both steps can only decrease SSE, and there are finitely many assignments — so no assignment can repeat without the procedure having stopped.

2. The mean is exactly the point minimising the **sum of squared** distances to a set — the update solves its half of the objective optimally.

3. The median minimises the sum of **absolute** distances, so you would be optimising a different, more outlier-robust criterion (k-medians).

k-means alternates **assign** and **move**, minimising SSE to a local optimum. The mean is not a convenience — it is the minimiser of the criterion, which is why changing the criterion changes the update.

Module 2

Mixture Models and EM

Guiding Question: *If the cluster label were just another unobserved variable, could we infer it the way we infer anything else?*

Soft Clustering

k-means assigns each point to exactly one cluster. But a point **between** two groups is not confidently in either — and the hard assignment throws that information away.

A **Gaussian mixture model** says each point was generated in two steps:

1. pick a component k with probability π_k ,
2. draw $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$.

$$p(\mathbf{x}) = \sum_{k=1}^K \pi_k \mathcal{N}(\mathbf{x} \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)$$

The component that generated each point is a **latent variable** — never observed.

Why This Needs a New Algorithm

If we knew the assignments, fitting each Gaussian would be trivial. If we knew the Gaussians, computing the assignments would be trivial. **We know neither.**

Expectation–Maximisation

E-step. Given the current parameters, compute the **responsibility** of component k for point i — the posterior over the latent variable:

$$\gamma_{ik} = \frac{\pi_k \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k)}{\sum_j \pi_j \mathcal{N}(\mathbf{x}_i \mid \boldsymbol{\mu}_j, \boldsymbol{\Sigma}_j)}$$

M-step. Given the responsibilities, re-estimate each $\pi_k, \boldsymbol{\mu}_k, \boldsymbol{\Sigma}_k$ by weighted maximum likelihood.

Repeat. The log-likelihood never decreases.

The E-step is **Bayes' theorem**: prior π_k times likelihood, normalised over components.

Nothing new is being invented.

The Relationship to k-Means

k-means

Hard assignment; spherical, equal-sized clusters implied by the SSE criterion; one label per point.

GMM with EM

Soft assignment (responsibilities); each component has its own **covariance**, so clusters may be elongated and differently sized; a full density model.

k-means is the limiting case of EM on a GMM with equal spherical covariances as the variance goes to zero — the responsibilities collapse to 0/1. The alternating structure is the same because it is the same idea.

Mini-Exercise

A point sits exactly between two fitted components.

1. What does k-means report?
2. What does the GMM report?
3. Give a decision where the difference matters.

Mini-Exercise — Solution

A mixture model makes the cluster assignment a **latent variable with a posterior**. EM alternates inferring it and re-fitting the parameters — and k-means is what remains when the uncertainty is squeezed out.

Mini-Exercise — Solution

1. One cluster label — arbitrary, and with no indication that it was close.

2. Responsibilities near 0.5/0.5: an explicit statement of ambiguity.

3. Any decision where being wrong is costly — route the ambiguous case to a human, or defer. That requires knowing it was ambiguous.

A mixture model makes the cluster assignment a **latent variable with a posterior**. EM alternates inferring it and re-fitting the parameters — and k-means is what remains when the uncertainty is squeezed out.

Module 3

Keeping the Uncertainty

Guiding Question: *Your model predicts €2.1M for a house four times larger than anything it was trained on. Should it say so?*

Keeping the Distribution

MAP estimation put a prior on the weights and then **collapsed it to a single point**. The Bayesian treatment keeps the whole distribution:

$$p(\mathbf{w} \mid D) = \frac{p(D \mid \mathbf{w})p(\mathbf{w})}{p(D)}$$

Prior

$p(\mathbf{w}) = \mathcal{N}(0, \tau^2 \mathbf{I})$ — before any data, small weights are more plausible.

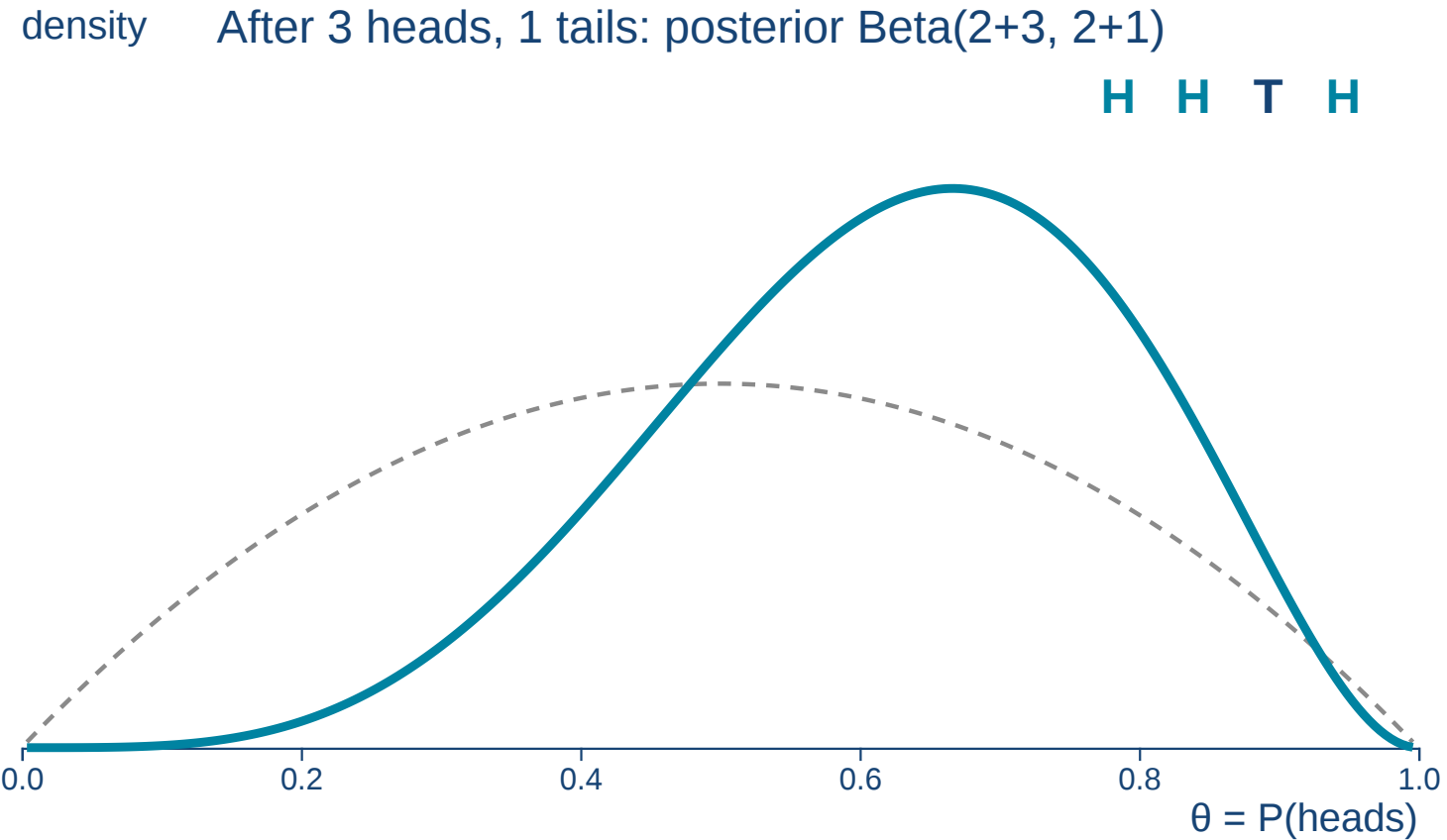
Likelihood

The Gaussian noise model from the linear-regression unit.

Posterior

Also Gaussian — the prior is **conjugate**, so the update has a closed form.

Watching the Posterior Concentrate



The same mechanism, shown for a single parameter: the prior is broad, each observation narrows it, and the posterior is what the next observation updates.

The Payoff: Predictions With Honest Error Bars

Instead of one prediction, integrate over every plausible parameter setting:

$$p(y^{\{ \mid \mathbf{x}^{\{ \}}, D) = \int p(y^{\{ \mid \mathbf{x}^{\{ \}}, \mathbf{w}) p(\mathbf{w} \mid D) d\mathbf{w}}$$

Near the data

Many plausible weight vectors agree, so the **predictive interval is narrow**.

Far from the data

Plausible weight vectors disagree wildly, so the **interval widens** — the model reports that it is extrapolating.

This is **epistemic** uncertainty, made operational. A point estimate — MLE or MAP — extrapolates with exactly the same confidence it interpolates, which is the failure mode behind most confidently wrong predictions.

Mini-Exercise

Two models are fitted to house prices for homes of 50–150 m². Both predict €2.1M for a 600 m² house.

1. Model A is OLS. What does it report about that prediction?
2. Model B is Bayesian. What would you expect it to report?
3. Which one would you put in front of a client?

Mini-Exercise — Solution

Bayesian regression replaces a **best parameter** with a **distribution over parameters**, and a prediction with a **predictive distribution**. The interval widens where the data is thin — which is exactly where a point estimate is most dangerous.

Mini-Exercise — Solution

1. A point estimate with the **same** residual-based interval as anywhere else — no signal that 600 m² is far outside the data.

2. A very wide predictive interval: the training data barely constrains the slope that far out.

3. B — not because the number is better (it is the same) but because it tells the client not to rely on it.

Bayesian regression replaces a **best parameter** with a **distribution over parameters**, and a prediction with a **predictive distribution**. The interval widens where the data is thin — which is exactly where a point estimate is most dangerous.